LCS - APA

Fabio Nicolosi s7592267@studenti.unige.it

May 2026

1 Sottosequenza

Per comprendere a fondo il problema della *Longest Common Subsequence* (LCS), è fondamentale avere ben chiaro il concetto di sottosequenza.
Una sottosequenza di una sequenza s è una successione di elementi presi dal testo di partenza nello stesso ordine, ma non necessariamente contigui. In breve, essa si ottiene eliminando zero o più elementi dalla sequenza originale, senza mai modificarne l'ordine relativo.

Esempio:
Sia $s =$ "ABCD"

- $s_1 =$ "ABD" è una sottosequenza di s
- $s_2 =$ "BD" è una sottosequenza di s
- $s_3 =$ "CAB" **NON** è una sottosequenza di s (l'ordine degli elementi è invertito)

Di conseguenza, il problema LCS consiste nel trovare, date due sequenze in input, una sottosequenza comune che abbia la massima lunghezza possibile.
Per risolvere questo problema si potrebbe utilizzare, in modo ingenuo, un approccio a forza bruta. Date due stringhe s_1 (di lunghezza m) e s_2 (di lunghezza n), l'algoritmo genera tutte le possibili sottosequenze di s_1. Poiché ogni elemento può essere incluso o escluso, si generano 2^m sottosequenze. Successivamente, per ciascuna di esse, si verifica se sia presente anche in s_2, tenendo traccia della più lunga individuata. Questa verifica richiede, nel caso peggiore, un numero di passaggi pari alla lunghezza di s_2 (n). Questo è un pessimo approccio, poiché ha un costo computazionale pari a $\Theta(n \times 2^m)$.

Esempio:
Se applicassimo la forza bruta a due stringhe con s_1 di 100 caratteri e s_2 di 200 caratteri, l'algoritmo richiederebbe circa 200×2^{100} operazioni, un numero talmente elevato da rendere il calcolo impraticabile.

2 Algoritmo

```cpp
int main() {
    string s1 = readfile("dna_sequence_100.txt");
    string s2 = readfile("dna_sequence_200.txt");

    unsigned int L[201][101];
    Direction R[201][101];

    for (unsigned int i = 0; i < 201; i++) L[i][0] = 0;
    for (unsigned int j = 0; j < 101; j++) L[0][j] = 0;

    for (unsigned int i = 1; i < 201; i++) {
        for (unsigned int j = 1; j < 101; j++) {
            if (s2[i-1] == s1[j-1]) {
                L[i][j] = L[i-1][j-1] + 1;
                R[i][j] = D;
            } else {
                R[i][j] = getDirection(L, i, j);
                if (R[i][j] == LE){

                    L[i][j] =  L[i][j-1];
                }
                else{
                    L[i][j] =  L[i-1][j];

                }
            }
        }
    }

    unsigned int dim = L[200][100];

    string seq;
    unsigned int i = 200, j = 100;

    while (i > 0 && j > 0) {
        if (R[i][j] == D) seq.push_back(s1[j-1]);
        move(R[i][j], i, j);
    }
```

Codice - C++

Come precedentemente illustrato, l'utilizzo della forza bruta per risolvere il problema della LCS comporta un costo computazionale molto elevato, rendendo l'algoritmo del tutto ina-

datto per sequenze con un grande quantitativo di caratteri. In alternativa, è possibile adottare un approccio molto più efficiente basato sulla **programmazione dinamica** con strategia bottom-up. Questa tecnica è ideale poiché i sottoproblemi affrontati – ovvero il confronto delle sottosequenze dei prefissi in entrambe le stringhe – tendono a ripetersi più volte. Di conseguenza, conviene risolvere ogni sottoproblema una sola volta e memorizzarne i risultati all'interno di apposite tabelle (matrici) per evitare ricalcoli inutili. Come si può osservare nel codice, vengono impiegate due matrici: la matrice L per le lunghezze (i *"pesi"*) e la matrice R per i riferimenti (*la direzione delle frecce*).

2.1 Logica induttiva

L'algoritmo confronta i prefissi delle due stringhe, che ipotizziamo essere s_1 di lunghezza m e s_2 di lunghezza n. Per calcolare il valore generico della cella (i, j) della matrice, si esaminano gli elementi correnti $s_1[j]$ e $s_2[i]$:

- **Caso base**: Se almeno una delle due sequenze è vuota (ovvero quando l'indice della riga $i = 0$ o della colonna $j = 0$), la lunghezza della LCS è inevitabilmente 0 e non è presente alcuna sottosequenza comune.
- **I caratteri coincidono** ($s_2[i-1] == s_1[j-1]$): Se i caratteri correnti sono uguali, significa che abbiamo trovato un elemento in comune. La lunghezza della LCS aumenta di 1 rispetto al valore memorizzato nella cella diagonalmente precedente. In questo caso, nella matrice dei riferimenti viene inserita una freccia diagonale (D)

$$\text{LCS}(i, j) = \text{LCS}(i-1, j-1) + 1$$

- I caratteri non coincidono ($X[i] \neq Y[j]$): Se i caratteri sono diversi, l'elemento corrente non può far parte della sottosequenza comune. Per procedere, si seleziona il valore massimo tra le soluzioni dei sottoproblemi contigui, ovvero la cella adiacente superiore o quella a sinistra. Nella matrice dei riferimenti si inserisce una freccia (verso l'alto $[U]$ o verso sinistra $[LE]$) rivolta verso la cella che conteneva la lunghezza maggiore:

$$\text{LCS}(i, j) = \max(\text{LCS}(i-1, j), \text{LCS}(i, j-1))$$

```
Direction getDirection(unsigned int l[201][101], int i, int j) {
    if (l[i][j-1] > l[i-1][j]) return LE;
    else return U;
}
```

2.2 Finale

Applicando queste regole induttive attraverso due cicli annidati, l'algoritmo compila l'intera matrice fino a raggiungere l'ultima cella in basso a destra, che conterrà la lunghezza della LCS ottimale. Per ricostruire la stringa effettiva, si esegue un percorso a ritroso partendo dall'angolo inferiore destro e seguendo le frecce memorizzate. Ogni volta che si incontra una freccia diagonale, si preleva il carattere corrispondente da una delle due stringhe. Poiché il tracciamento avviene dalla fine all'inizio, gli elementi verranno estratti in ordine inverso rispetto a quello corretto; sarà quindi necessario invertire la stringa risultante per ottenere la LCS definitiva.

```
void move(Direction mv, unsigned int &i, unsigned int &j) {
    switch (mv) {
        case U:  i--; break;
        case LE: j--; break;
        case D:  i--; j--; break;
        default: break;
    }
}
```

3 Dati

3.1 Input - Fibonacci

```
0, 1, 1, 2, 3, 5, 8, 13, 21, 34
```

```
0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89
```

3.2 Output - Fibonacci

```
LCS: 0, 1, 1, 2, 3, 5, 8, 13, 21, 34
Lunghezza: 31
```

Attezione: in questo caso vediamo che la lunghezza è di *31*, questo è dovuto a tutti i caratteri come: *lo spazio* e *le virgole*, che in C++ vengono contati.

3.3 Input - DNA

```
GCGTCCCCTCCATCCTTACCGCACCCCTACGAGGTTCATCACCCACCCATTGCCAGCTGTACCTGCAGTGACAAAGCCATAACCCCGACTAGAAAGGCAC
```

```
CGATTGATATAGGCACTCTGGGTTTGGACACCGACTATTGTCTAGCCTCCTATGTTACAAGGCAGTAACCACACTCTACAACTTAACACAAACCATGCG
AAAGTGAATGTGGAGCCGTTCCCTACTACACTTCCGATGCTGACGCTACTCTCATCTACGGTGCCCGTACGAAAGCAGTGACGAACCTGGTCGAACAGCAG
```

3.4 Output - DNA

```
LCS: GCGTCCCCTCATCCTTACCGCACCCCTACGAGGTTACCCCCCCATTCCAGCTGTACCTCAGTGACAAAGCATACGACTGAAAGCA
Lunghezza: 85
```

4 Codice

```
enum Direction { U, LE, D };

Direction getDirection(unsigned int l[201][101], int i, int j) {
    if (l[i][j-1] > l[i-1][j]) return LE;
    else return U;
}

void move(Direction mv, unsigned int &i, unsigned int &j) {
    switch (mv) {
        case U:  i--; break;
        case LE: j--; break;
        case D:  i--; j--; break;
        default: break;
    }
}

string readfile(const string &fileName) {
    ifstream file(fileName);
    if (!file.is_open()) {
        cerr << "Errore: impossibile aprire il file " << fileName << endl;
        return "";
    }
    ostringstream buffer;
    buffer << file.rdbuf();
    return buffer.str();
}

int main() {
    string s1 = readfile("dna_sequence_100.txt");
    string s2 = readfile("dna_sequence_200.txt");

    unsigned int L[201][101];
    Direction R[201][101];

    for (unsigned int i = 0; i < 201; i++) L[i][0] = 0;
    for (unsigned int j = 0; j < 101; j++) L[0][j] = 0;

    for (unsigned int i = 1; i < 201; i++) {
        for (unsigned int j = 1; j < 101; j++) {
            if (s2[i-1] == s1[j-1]) {
                L[i][j] = L[i-1][j-1] + 1;
                R[i][j] = D;
            } else {
                R[i][j] = getDirection(L, i, j);
                if (R[i][j] == LE){

                    L[i][j] =  L[i][j-1];
                }
                else{
                    L[i][j] =  L[i-1][j];

                }
            }
        }
    }

    unsigned int dim = L[200][100];

    string seq;
    unsigned int i = 200, j = 100;

    while (i > 0 && j > 0) {
        if (R[i][j] == D) seq.push_back(s1[j-1]);
        move(R[i][j], i, j);
    }

    reverse(seq.begin(), seq.end()); //sequenza era invertita

    cout << "LCS: " << seq << "\nLunghezza: " << dim << endl;

    // Scrittura su file di output
    ofstream out("lcs_output.txt");
    out << "LCS: " << seq << "\nLunghezza: " << dim << endl;
    out.close();

    return 0;
}
```